

Response to the independent R36 referee report

Anonymous revision R37, September 24, 2026

Manuscript and report identities

Revised title: **Service Contracts with Limited Memory: Participation, Randomization, and Exact Design.**

This response addresses reviews/operation_research_referee_report_r36_independent_harsh_2026-09-24.md on review commit f26f4da71207a7613735b0504bc34dbe231813c1. That report reviewed R36 at 8b17bed9079aa8e3bc5f52e6a5847a60cbf7d7ca. It is not a response to the earlier R30 report. The revision branch starts from the latest review commit, preserving both the reviewed manuscript and the report in ancestry.

We have reconstructed the article around the **restricted-memory service-design frontier**. The central claims remain the globally optimized continuous codebook, the institutional separation between expected and pathwise participation, and the contractual cost structure that permits exact complete frontiers. The broader compact-response and exact-memory results remain available in the companion and preservation archive, rather than being withdrawn. The new title, introduction, ordering of results, and conclusion follow this single decision problem. The proposed review area is Optimization.

1. Partial and staircase Monge priority

The report correctly identifies a comparison that cannot be resolved by citing full-matrix SMAWK alone. The revised related-work section and priority table distinguish four layers: full totally monotone search; partial and staircase search; Monge dynamic-programming optimization; and the contractual finite-cost structure.

We add Klawe and Kleitman (1990), Kaplan, Mozes, Nussbaum, and Sharir (2017), Chan (2021), and Emmerich (2026), while retaining Aggarwal et al. (1987) and Yao (1980). The crucial comparison is stronger than a generic acknowledgment. Chan’s author version, pages 2–3, explicitly distinguishes staircase orientations permitting direct completion and SMAWK from the harder orientations. The terminal array here has shrinking feasible suffixes and belongs to the easy case under the stated concave-Monge convention. Reversing both orders of the expanding-prefix program arrays places them in the same case.

Accordingly, the article does not claim a new general completion primitive, a general partial-Monge algorithm, or an improvement over the generic inverse-Ackermann bound. Its mathematical content is the contractual cap-anchoring reduction, the off-cap optimum, and the preservation of Monge structure after continuous terminal minimization. The latter uses a specific difference identity in which heterogeneous intermediate curvatures cancel. The linear complete-frontier theorem then follows by an explicitly verified application of classical searching. Emmerich’s different Monge-DP application is discussed as evidence that the broad algorithmic progression is established, not as an algorithm solving these contract costs.

Locations: `positioning.tex`, `monge_completion.tex`, and the theorem-by-theorem priority table. No economic or global-optimality conclusion is removed by this attribution.

2. A formal completion proof, including ties and infeasible submatrices

The main article now defines the exact strict preference implication used by the implementation. Finite entries have keys $(0, \text{cost}, \text{column})$, so finite equality selects the left column. The suffix and prefix completions are separate lemmas with different padding orders: $(1, 0, -\text{column})$ for infeasible left entries of a shrinking suffix, and $(1, 0, \text{column})$ for infeasible right entries of an expanding prefix.

The suffix proof explicitly treats finite/finite, finite-to-padded, mixed feasible/padded, and padded/padded cases. The prefix proof establishes why an upper-row preference for the right column can only arise from two finite entries, which remain finite later. Both proofs cover selected submatrices with no feasible entry in a row and identify the appropriate extreme selected column. A counterexample shows why uniform infinite padding with ordinary left ties is insufficient. Another shows that a generic two-bound helper is not a theorem for arbitrary interval domains.

A separate column-offset lemma proves that previous-layer values preserve the finite Monge inequality. A reachability lemma proves the exact predecessor sets: the initial singleton, followed by a contiguous interval of reachable endpoints. Unreachable predecessors are removed, never inserted as finite column costs. The main text supplies pseudocode for the terminal, anchored-prefix, and deterministic-prefix interfaces, including winning-cell reevaluation and reconstruction.

The new exact suite checks every selected nonempty row/column submatrix through dimension four for both orientations, ties, and signed column offsets. It also substitutes independently authored PADS search. These tests audit implementation cases; the all-size result rests on the displayed proofs.

3. One central article instead of a cumulative research catalogue

We adopt the report's restricted-memory-frontier identity. The article now begins with a renewal desk passing a limited service code to a common execution system. It establishes the deterministic frontier, explains the expected-participation lottery, solves the joint continuous codebook problem, proves the pathwise separation, and derives the exact matrix-search consequence. Resource pricing and implementation evidence then answer the design question introduced at the beginning.

Compact response compilation, tight response size, piecewise-quadratic closure, general behavioral minimization, radial dispersion, and the shared-table comparator no longer compete as separate headline contributions. Their assumptions and proofs are preserved. The source preservation map records where each inherited module is currently read and where its exact preceding presentation remains available. Original theorem modules and historical evidence are not overwritten or silently shortened. The exact R36 main and companion readers are archived for audit.

The conclusion now has two takeaways: participation timing changes the value of retained information, and an exact loss frontier must be combined with explicit implementation costs. It no longer recites every revision's theorem family.

4. Operational institution and the meaning of a private draw

The new contract section states the saturated root promise before using it. Positive branch probabilities and the root equality force each expected cap to bind; under pathwise caps the same argument forces realization-by-realization equality. These are used as substantive hypotheses, not normalizations hidden in a later proof.

A stipulated renewal-desk/dispatch-gateway mechanism states who sees the branch, which record crosses the execution boundary, when an offer and its lottery are communicated, when the customer may exit, and how the service ticket is implemented. The customer approves before the ticket draw. A later realized ticket need not remain secret, but the episode does not introduce a new costless exit after its disclosure. The retained symbol, not a free shared seed or recoverable branch identifier, is the decoder’s historical input.

An expected-obligation cap is explicitly distinguished from an ex post billing, safety, or enforceable realization cap. We make no legal enforceability claim. A hard realization constraint requires the pathwise frontier. Repeated customers who may condition later acceptance on past draws require an expanded public state or charged history and are not automatically covered by the three-date theorem. A contractible seed disclosed before acceptance changes the conditioning information; it cannot be introduced as free controller memory.

This is a documented stylized mechanism, not invented operational data or an empirical calibration. The report explicitly allowed a carefully specified stylized mechanism as an alternative to calibration, and the revised claims are tied to that mechanism.

5. An intermediate institution, not only the two endpoints

The revision also analyzes a bounded-overflow agreement: the intermediate tier is fixed before the ticket draw, expected total payment equals the branch cap, and every realized total is at most the cap plus a stated tolerance. The pre-draw intermediate-action requirement is explicit; we do not assert that it is without loss for all bounded-risk models.

For any fixed codebook, the article appendix proves the exact branch rule. A branch uses its adjacent mean-preserving lottery only when the upper codeword’s overflow fits the tolerance; otherwise it uses the highest codeword below its cap deterministically. The proof excludes any positive probability on an inadmissible realization rather than replace the hard requirement with an expectation.

For the existing heterogeneous three-branch off-cap example, the global two-code optimum is $(1/4, z)$, with $z = 1/2 + \min(\text{tolerance}, 1/8)$. Its exact loss is $z^2/20 - z/16 + 3/80$. This connects the deterministic loss $3/160$ to the expected-participation loss $23/1280$ and identifies the realized high-draw overflow and its probability. The proof optimizes over all two-level codebooks; a nonanchor grid is only a regression check. We do not claim the general bounded-overflow problem inherits the full cap-anchoring or linear-search theorem.

6. Integrated implementation-resource accounting

The new resource section minimizes service loss plus priced writable bits, specified read-only storage, evaluation work, and amortized compilation, with an optional participation-protocol charge. Writable capacity is measured as $\text{ceil}(\log_2 m)$ bits, not m bits. Decoder levels, branch encoders, public observations, coefficient precision, and read-only programs are separated.

The optimization is exact for the stated implementation menu, whose representation charges are fixed by capacity and representation choice. We also identify the boundary: when read-only cost depends on the particular codebook, a loss-minimizing codebook alone need not minimize the joint cost. The one-dimensional frontier is then a primitive, not an unsupported solution to every multidimensional controller-design problem.

The equal-probability three-branch example gives a concrete capacity decision. For every bit price strictly between $1/96$ and $1/8$, expected participation selects one writable bit while pathwise participation selects two. The zero-bit alternative is checked explicitly. This turns the institutional loss difference into a resource-allocation prescription.

7. Practical computation rather than an asymptotic speed claim

The current study uses fresh processes, computes every budget through eight, and extends the same rational input family through 16,384 branches. Each process has a stated 120-second limit. An incomplete case is recorded as a limit observation, not a fabricated runtime. Results retain unfavorable cases and distinguish economic requests, wall time, resident memory, and Python-tracked allocation peaks.

The PADS offline function is independently authored and MIT licensed; its source commit, original blob, excerpt scope, and license are recorded. Only the search layer is substituted. Shared economic oracles and reconstruction are disclosed. Separate cProfile runs decompose exclusive work, and the combined scan/reconstruction category is not mislabeled as isolated reconstruction time.

A second external comparison assembles the reduced finite networks directly from rational branch sums and solves them with compiled HiGHS through SciPy. The highest service level is still minimized continuously, not forced onto a cap grid. Construction, floating-point solver time, and exact certification/replay are recorded separately. Each returned path is repriced in exact fractions and certified with exact shortest-path potentials. This is meaningful external computation on the proved finite reduction, not an independent proof of the continuous reduction itself.

Python Fraction timings serve exact audit and implementation comparison. They are not claimed to predict an optimized native matrix-search implementation, and the compiled LP comparison is not relabeled as native SMAWK. The numerical section uses the executed measurements below; earlier results remain historical rather than being mixed into the current timing sample.

8. One complexity account

The main theorem and its accounting table now state the sorted-input interface, optional sorting cost, moment construction, exact interval minimization, repeated feasible-cost evaluation, padding, backpointers, every budget's reconstructed codebook, output writing, rational heights, and integer/gcd costs in one place. Public model data and external solver preprocessing are explicitly distinguished from the renewal search.

The proof explains why searching introduces no new algebraic depth: a queried entry is one of the same finite candidates available to quadratic enumeration, with at most a previous path of anchored costs plus one continuously minimized tail. Repetition affects the number of operations, not the rational-height dependence. Padding uses only index keys. Both rational-operation and conservative bit bounds include comparisons and output, and the abstract names ordered rational inputs.

9. Editorial identity, presentation, and current package

The title, abstract, equation-free introduction, and proposed review area now identify an Optimization paper about service-code design. The manuscript uses an anonymous title page, an abstract under 200 words, 11-point type, 1.5 spacing, one-inch margins, author-year references, and tables after references. The build records actual page counts and checks the applicable main/companion limits; it does not infer format compliance from the template name.

Readers start with `root main.pdf` and `electronic_companion.pdf`. The current code is under the `R37 code/` directory, with one `reproduce.py` entry point, and one current build manifest binds sources, tests, measurements, and readers. The preservation map and preceding-reader archive serve historical audit and do not compete as current manuscript versions. No base theorem or evidence file is deleted. No other review, revision, or main branch is updated by the publication workflow.

Further review

The revised publication case does not depend on claiming that classical staircase search is new or that a synthetic study is an empirical deployment. It rests on the global continuous memory-design result, the sharp participation-dependent frontier, and the explicit resource decision they support. The completion and complexity proofs are now matched to the implementation, and the comparison package tests the residual algorithmic claims with independent search and external optimization components. The executed evidence and build identities are recorded in the accompanying current manifest for the next referee.

Executed evidence

The executed suite checks 784 completed matrices and 282,832 selected-submatrix row minima, including 98,728 wholly padded selected rows. It records 1,890 exact frontier equalities and direct controller replays, 190 bounded-overflow all-pairs equalities, and 33 evaluations of the solved tolerance frontier. The unchanged

R33–R36 suites also pass in isolated directories. All 24 independently assembled HiGHS cases yield an exactly certified reduced-network optimum and a matching controller.

At the largest completed common size for expected participation, 16,384 branches, the retained SMAWK implementation takes 26.601 seconds against 37.436 for divide-and-conquer, a time ratio of 0.711. At the largest completed common size for pathwise participation, 16,384 branches, the retained SMAWK implementation takes 44.132 seconds against 52.054 for divide-and-conquer, a time ratio of 0.848. Across the 6 completed same-size comparisons, SMAWK is slower in 2. These are observations on one runner and one input family, not a uniform speedup claim. No configured study case exceeded the stated process limit.

The machine-readable evidence is in the current results directory. The current build manifest records the exact scientific source commit, PDF hashes, page counts, and base-tree preservation audit.